

Projet Notebook Python

Algèbre linéaire et systèmes dynamiques

Application du théorème de Perron–Frobenius

Objectif général. On étudie un système dynamique linéaire à trois états. L’objectif est de déterminer, à l’aide de calculs théoriques puis de simulations numériques, si la suite des vecteurs de proportions $(v_t)_{t \geq 0}$ converge vers un même vecteur limite v_∞ , indépendamment de la répartition initiale. Le projet doit faire apparaître le lien entre la structure spectrale de la matrice de transition, le théorème de Perron–Frobenius et le comportement à long terme du système.

Travail attendu

Le rendu principal est un Notebook Jupyter au format `.ipynb`. Il doit alterner des cellules Markdown, contenant les raisonnements mathématiques et les interprétations, et des cellules Python, contenant le code, les tests, les tableaux et les graphiques.

Le Notebook doit pouvoir être exécuté intégralement, dans l’ordre, après un redémarrage du noyau. Les résultats numériques seuls ne constituent pas une justification des questions théoriques de la partie I.

Contexte — Mobilité résidentielle entre trois zones

Une population se répartit entre trois zones résidentielles :

E_1 : zone urbaine, E_2 : zone périurbaine, E_3 : zone rurale.

À l’instant t , on note

$$v_t = \begin{pmatrix} x_t \\ y_t \\ z_t \end{pmatrix}, \quad x_t + y_t + z_t = 1, \quad x_t, y_t, z_t \geq 0,$$

où x_t , y_t et z_t désignent les proportions d’individus résidant respectivement dans les zones E_1 , E_2 et E_3 . On suppose que les mobilités observées d’une période à la suivante sont les suivantes :

- parmi les habitants de la zone urbaine, la moitié y reste, un quart rejoint la zone périurbaine et un quart rejoint la zone rurale ;
- parmi les habitants de la zone périurbaine, les deux tiers y restent, un sixième rejoint la zone urbaine et un sixième rejoint la zone rurale ;
- parmi les habitants de la zone rurale, les deux tiers y restent, un sixième rejoint la zone urbaine et un sixième rejoint la zone périurbaine.

Les coefficients sont supposés constants au cours du temps. On adopte la convention suivante : le coefficient a_{ij} représente la proportion des individus provenant de la zone E_j qui se trouvent dans la zone E_i à la période suivante.

1. Mise en équations du système

- a) Écrire les trois relations donnant x_{t+1} , y_{t+1} et z_{t+1} en fonction de x_t , y_t et z_t .
- b) En déduire une écriture matricielle de la forme

$$v_{t+1} = Av_t,$$

où $A \in \mathcal{M}_3(\mathbb{R})$. Expliciter entièrement la matrice A .

- c) Montrer par récurrence que, pour tout entier $t \geq 0$,

$$v_t = A^t v_0.$$

2. Propriétés stochastiques

- a) Rappeler la définition d'une matrice stochastique par colonnes, puis vérifier que A possède cette propriété.
- b) Montrer que si v_t est un vecteur de proportions, alors v_{t+1} est encore un vecteur de proportions. Interpréter ce résultat dans le contexte de l'étude.

3. Mise en évidence de la valeur propre 1

On note

$$\mathbb{1} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.$$

- a) Calculer $A^\top \mathbb{1}$. En déduire une valeur propre pour $A^\top$.
- b) Justifier soigneusement que $\text{Sp}(A) = \text{Sp}(A^\top)$. Conclure que 1 est une valeur propre de A .
- c) Déterminer une base de $\text{Ker}(A - \text{Id}_3)$. En déduire un vecteur propre π associé à la valeur propre 1, à coefficients positifs et normalisé par la condition

$$\pi_1 + \pi_2 + \pi_3 = 1.$$

Quelle est l'interprétation de π dans le contexte du problème ?

4. Détermination du spectre

- a) Calculer $\text{tr}(A)$ et $\det(A)$.
- b) On note 1, λ_2 et λ_3 les trois valeurs propres de A . En déduire λ_2 et λ_3 et donner le spectre $\text{Sp}(A)$.
- c) Vérifier le résultat en calculant et en factorisant le polynôme caractéristique de A .

5. Diagonalisation

- a) Justifier que A est diagonalisable sur $\mathbb{R}$.
- b) Pour chaque valeur propre de A , déterminer une base du sous-espace propre associé.
- c) Construire une matrice inversible P et une matrice diagonale D telles que

$$A = PDP^{-1}.$$

L'ordre des colonnes de P devra être cohérent avec celui des valeurs propres placées sur la diagonale de D .

6. Application du théorème de Perron–Frobenius

Rayon spectral

On définit le **rayon spectral** d'une matrice carrée A quelconque comme le plus grand des modules de ses valeurs propres :

$$\rho(A) = \max\{|\lambda|, \lambda \in \text{Sp}(A)\}.$$

Théorème de Perron–Frobenius (*version admise*)

Soit B une matrice carrée réelle dont tous les coefficients sont strictement positifs. Son rayon spectral $\rho(B)$ est une valeur propre réelle strictement positive. Cette valeur propre est simple et possède un vecteur propre à coefficients strictement positifs, unique à multiplication par un scalaire près. Toute autre valeur propre λ de B vérifie

$$|\lambda| < \rho(B).$$

- a) Justifier que le théorème de Perron–Frobenius s'applique à A .
- b) Expliquer pourquoi le vecteur stationnaire π (*i.e.*, celui vérifiant $\pi = A\pi$) est unique parmi les vecteurs de proportions.

7. Puissances de A et comportement asymptotique

- a) Établir, pour tout entier $n \geq 0$, la relation $A^n = PD^nP^{-1}$. En déduire une expression explicite de A^n .
- b) On considère l'état initial

$$v_0 = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{4} \\ \frac{1}{4} \end{pmatrix}.$$

Calculer explicitement $v_t = A^t v_0$, puis déterminer $v_\infty = \lim_{t \rightarrow +\infty} v_t$.

- c) Déterminer la limite $A_\infty = \lim_{n \rightarrow +\infty} A^n$.
- d) Soit maintenant v_0 un vecteur de proportions quelconque. Montrer que $A^t v_0$ converge encore vers le même vecteur π . Répondre ainsi à la question générale posée au début du projet.

Rappel — Norme de Frobenius

La partie II mentionne la norme de Frobenius, qui permet notamment de munir l'espace des matrices d'une topologie naturelle. Pour une matrice réelle $M = (m_{ij})_{ij} \in \mathcal{M}_{p,q}(\mathbb{R})$, la norme de Frobenius est définie par

$$\|M\|_F = \sqrt{\text{tr}(M^\top M)} = \sqrt{\sum_{i=1}^p \sum_{j=1}^q m_{ij}^2}.$$

Ainsi, l'écart entre deux matrices A et B de mêmes dimensions est mesuré par

$$\|A - B\|_F = \sqrt{\sum_{i=1}^p \sum_{j=1}^q (a_{ij} - b_{ij})^2}.$$

Partie II — Implémentation numérique dans un Notebook 10pts

On pourra utiliser les bibliothèques `numpy`, `matplotlib` et, si nécessaire, `pandas`. L'emploi de `scipy` est autorisé, mais les fonctions utilisées devront être brièvement documentées dans le Notebook.

1. Construction et validation de la matrice de transition

- Construire la matrice A sous la forme d'un tableau NumPy de nombres flottants. Afficher sa forme, son type et son contenu.
- Programmer la fonction suivante :

```
def is_stochastic(A, orientation="column", atol=1e-12):  
    """Teste si A est stochastique par colonnes ou par lignes."""
```

La fonction doit notamment vérifier que A est carrée, que ses coefficients sont positifs ou nuls à la tolérance près, et que les sommes appropriées valent 1 à la tolérance près.

- Tester la fonction sur la matrice A , sur A^T et sur au moins une matrice qui n'est pas stochastique. Commenter les résultats.

2. Simulation du système dynamique

Programmer une fonction respectant l'interface suivante :

```
def simulate_markov(A, x0, T):  
    """Retourne les états x_0, x_1, ..., x_T du système x_{t+1}=A x_t."""
```

La fonction devra :

- vérifier la compatibilité des dimensions ;
- inclure l'état initial dans le résultat ;
- retourner un tableau de taille $(T + 1) \times 3$, dont la ligne d'indice t contient le vecteur d'état à l'instant t ;
- effectuer la simulation par récurrence, sans appeler directement une fonction de puissance matricielle.

Tester la fonction pour les états initiaux suivants :

$$v_0^{(1)} = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{4} \\ \frac{1}{4} \end{pmatrix}, \quad v_0^{(2)} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad v_0^{(3)} = \begin{pmatrix} \frac{1}{7} \\ \frac{2}{7} \\ \frac{4}{7} \end{pmatrix}, \quad v_0^{(4)} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

3. Représentation graphique des trajectoires

Pour chacun des états initiaux précédents, représenter sur un même graphique les trois suites (x_t) , (y_t) et (z_t) . Les figures devront comporter :

- un titre indiquant l'état initial ;
- les légendes des trois composantes ;
- les noms des axes ;
- une grille lisible ;
- des droites horizontales indiquant les composantes théoriques du vecteur stationnaire π .

Vérifier numériquement, à chaque instant, que les coordonnées restent positives ou nulles et que leur somme reste égale à 1. Commenter les trajectoires observées et comparer les différents états initiaux.

4. Calcul numérique des valeurs et vecteurs propres

- a) Écrire une fonction utilisant le paquet de votre choix et retournant les valeurs propres de A ainsi qu'un vecteur propre associé à chacune d'elles. Par exemple :

```
def eigen_data(A):  
    """Retourne les valeurs propres et une matrice de vecteurs propres."""
```

- b) Ordonner les valeurs propres de manière cohérente et utiliser `numpy.real_if_close` si de très petites parties imaginaires dues aux erreurs d'arrondi apparaissent.
- c) Normaliser le vecteur propre associé à la valeur propre 1 pour que la somme de ses coordonnées soit égale à 1. Le comparer au vecteur théorique π .
- d) Comparer toutes les valeurs propres numériques avec les valeurs obtenues dans la partie I.

5. Diagonalisation numérique

Programmer une fonction de la forme

```
def diagonalize(A, atol=1e-12):  
    """Retourne P et D lorsque A est numériquement diagonalisable."""
```

La fonction devra construire P et D à partir des données propres, vérifier que P est inversible, puis retourner les deux matrices dans un ordre cohérent. Vérifier numériquement la relation

```
np.allclose(A, P @ D @ np.linalg.inv(P), atol=1e-10)
```

Comparer la diagonalisation numérique obtenue avec celle de la partie théorique. Expliquer pourquoi les vecteurs propres retournés par le logiciel peuvent différer des vecteurs propres calculés à la main sans que les résultats se contredisent.

6. Trois méthodes de calcul de A^n

Implémenter et comparer les trois méthodes suivantes.

Méthode 1 — Produit matriciel itératif

Écrire une fonction qui initialise le résultat à la matrice identité et effectue n multiplications successives par A :

```
def power_iterative(A, n):  
    """Calcule  $A^n$  par multiplications successives."""
```

La fonction devra également traiter correctement le cas $n = 0$.

Méthode 2 — Utilisation de la diagonalisation

À partir de $A = PDP^{-1}$, programmer :

```
def power_diagonalized(P, D, n):  
    """Calcule  $P D^n P^{-1}$ ,  $D$  étant diagonale."""
```

Le calcul de D^n devra exploiter directement le fait que D est diagonale.

Méthode 3 — Fonction NumPy

Utiliser directement :

```
np.linalg.matrix_power(A, n)
```

Comparaison deux à deux

Pour n variant de 0 à une valeur maximale $N_{\max}$ pertinente, calculer les trois écarts suivants avec la norme de Frobenius :

$$\begin{aligned}\varepsilon_{\text{it,diag}}(n) &= \|A_{\text{it}}^n - A_{\text{diag}}^n\|_F, \\ \varepsilon_{\text{it,np}}(n) &= \|A_{\text{it}}^n - A_{\text{np}}^n\|_F, \\ \varepsilon_{\text{diag,np}}(n) &= \|A_{\text{diag}}^n - A_{\text{np}}^n\|_F.\end{aligned}$$

Pour chacune des trois comparaisons :

- déterminer la première valeur de n pour laquelle l'écart est inférieur à 10^{-6} ;
- vérifier si cet écart reste ensuite inférieur à 10^{-6} sur tout l'intervalle étudié ;
- représenter l'écart en fonction de n avec une échelle verticale logarithmique ;
- commenter l'origine des éventuelles différences numériques.

Si le seuil est atteint dès $n = 0$ ou $n = 1$, il faut l'indiquer et l'expliquer : les trois méthodes sont mathématiquement équivalentes, et cette question mesure leur accord numérique, non la vitesse de convergence du système.

7. Convergence numérique vers la matrice limite

Afin d'étudier cette fois la convergence du système, utiliser la matrice A_∞ déterminée dans la partie I et définir

$$e(n) = \|A^n - A_\infty\|_F.$$

- Représenter $e(n)$ en fonction de n sur une échelle logarithmique.
- Déterminer numériquement le plus petit entier n tel que

$$e(n) < 10^{-6}.$$

- Comparer ce résultat à une estimation théorique fondée sur les valeurs propres de A .

8. États limites obtenus à partir de plusieurs conditions initiales

Pour chacun des états initiaux de la question 2, calculer une approximation de

$$v_\infty = \lim_{t \rightarrow +\infty} A^t v_0.$$

Présenter les résultats dans un tableau contenant au minimum : l'état initial, l'état final approché, la somme des coordonnées de cet état final et son écart au vecteur théorique π . Répondre ensuite aux questions suivantes.

- Les simulations suggèrent-elles l'existence d'un état stationnaire ?
- Cet état stationnaire semble-t-il dépendre de l'état initial ?
- Quel est le rôle particulier de la valeur propre $\lambda = 1$?
- Expliquer pourquoi on considère le module des autres valeurs propres. Rappeler également pourquoi toute valeur propre λ d'une matrice stochastique vérifie

$$|\lambda| \leq 1,$$

puis expliquer pourquoi l'inégalité est stricte ici lorsque $\lambda \neq 1$.

- Donner une interprétation concrète du vecteur limite dans le contexte de la mobilité résidentielle.

À ne pas confondre

Un *vecteur stationnaire* π vérifie $A\pi = \pi$. La convergence de $A^t v_0$ vers π est une propriété supplémentaire. Dans ce projet, elle résulte à la fois de la structure spectrale explicite de A et du théorème de Perron–Frobenius.

Bonus

+2pts

Bonus 1 — Comparaison des temps de calcul

À l'aide du module `time`, et de préférence de la fonction `time.perf_counter`, comparer les temps de calcul des trois méthodes de la question 6 pour plusieurs valeurs de n , par exemple

$$n \in \{10, 100, 1000, 10000\}.$$

Chaque mesure devra être répétée plusieurs fois ; on reportera, par exemple, la médiane des temps obtenus. Présenter les résultats dans un tableau clair, créé éventuellement avec `pandas`. Pour que la comparaison soit équitable, préciser si le temps nécessaire au calcul de P , D et P^{-1} est inclus ou non dans le chronométrage. Commenter les complexités et les limites de cette expérience pour des matrices de taille 3×3 .

Bonus 2 — Rôle de la deuxième valeur propre dominante

On appelle *deuxième valeur propre dominante* une valeur propre de module maximal parmi celles qui sont différentes de 1.

- Expliquer pourquoi son module gouverne en général la vitesse asymptotique de convergence vers l'état stationnaire.
- Relier cette valeur propre à la pente observée sur le graphique semi-logarithmique de $e(n)$.
- Montrer qu'un état initial particulier peut avoir une composante nulle suivant le sous-espace propre correspondant. Expliquer alors pourquoi la convergence peut être plus rapide pour cet état initial que pour un état initial générique.

Organisation conseillée du Notebook

Le Notebook pourra suivre l'organisation suivante :

- titre, noms des auteurs et présentation du problème ;
- calculs théoriques rédigés en Markdown avec formules LaTeX ;
- importation des bibliothèques et construction de A ;
- fonctions de validation et de simulation ;
- graphiques des trajectoires ;
- calculs spectraux et diagonalisation numérique ;
- comparaison des trois méthodes de calcul de A^n ;
- étude de la convergence et tableaux de résultats ;
- conclusion mathématique et interprétation dans le contexte.

Critères de qualité. La notation prendra en compte la justesse des calculs, la rigueur des justifications, la qualité et la réutilisabilité du code, la présence de tests, la lisibilité des graphiques, la pertinence des commentaires et la capacité à relier les résultats numériques aux propriétés théoriques. **Soignez en particulier la rédaction des résultats mathématiques.**